

“Public Transit Planner”

A Project Report submitted for the partial fulfilment of grade for the course

DATA STRUCTURES AND ALGORITHMS-3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

A.Y. 2026–2027

By

KARTHEEK REDDY 2520090015

DANUSH RAJ 2520080046

Team Number: 30 | Section: 10

Under the Esteemed Guidance of

Mrs. Chandusha Kanda

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Public Transit Planner”** is a record of bonafide work of **KARTHEEK REDDY – 2520090015, DANUSH RAJ – 2520080046** submitted in partial fulfilment of the requirements of the course DATA STRUCTURES AND ALGORITHMS-3 (25CS2103E), Section 10, Team 30, during the Academic Year 2026–2027, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

KARTHEEK REDDY – 2520090015

DANUSH RAJ – 2520080046

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Public Transit Planner”** is a bonafide work done and submitted by **KARTHEEK REDDY – 2520090015, DANUSH RAJ – 2520080046** in partial fulfilment of the requirements of the course **DATA STRUCTURES AND ALGORITHMS-3 (25CS2103E)**, Section 10, Team 30, at KLH University during the **Academic Year 2026–2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help and guidance of many people. We would like to express our sincere thanks to everyone who assisted us, in one way or another, in completing the Public Transit Planner project.

Our greatest appreciation goes to our Course Coordinator and to our guide, **Mrs. Chandusha Kanda**, Assistant Professor, Department of Computer Science and Information Technology, whose tremendous support, encouragement, and guidance carried this project from a rough idea to a working system.

We express our gratitude to **Dr. Kaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing the facilities, ways, and means by which we were able to complete this project.

We thank all the teaching and non-teaching staff members who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help and cooperation throughout our work.

KARTHEEK REDDY – 2520090015

DANUSH RAJ – 2520080046

Abstract

The Public Transit Planner is a data driven system designed to optimize public transportation routes and timetables while operating within fixed fleet and budget constraints. The project begins by collecting and cleaning transit stop, passenger demand, and travel time data to ensure accuracy and consistency. Using graph based routing and optimization techniques, the system identifies efficient routes that maximize service coverage and minimize passenger travel time and operating costs. It also generates practical timetables based on demand patterns and vehicle availability.

The planner evaluates performance using key metrics such as route coverage, average waiting time, travel time, fleet utilization, and operational expenses. Interactive reports and visualizations help compare different planning scenarios and support informed decision making. This project demonstrates the application of data preprocessing, optimization algorithms, and data analytics in addressing real world transportation challenges.

The proposed system improves accessibility, enhances service efficiency, and provides a cost effective decision-support solution for modern public transit planning.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Libraries	9
3	Technology Stack	3.1 Programming Language 3.2 Data Processing Libraries 3.3 Graph and Optimization 3.4 Visualization and Reporting 3.5 Version Control	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Processing Pipeline 4.4 Deployment Architecture	11
5	Design	5.1 Data Design Principles 5.2 Data Model 5.3 Algorithm Design 5.4 Constraints and Objective 5.5 Design Decisions and Assumptions	13
6	Implementation	6.1 Module-wise Implementation 6.2 Data Preprocessing 6.3 Graph Construction and Routing 6.4 Route Selection and Fleet Allocation 6.5 Timetable Generation 6.6 Metrics Computation 6.7 Visualization Logic	16
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	18
8	Testing	8.1 Testing Strategy 8.2 Unit and Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases	19
9	Deployment	9.1 Steps to Run 9.2 Environment Configuration 9.3 Hosting	21
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations	22
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	23
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	24
13	References	– Documentation, tools, and resources used	25
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Input Data Dictionary – Module Reference Summary – Screenshots (to be added by the student)	26

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Public transportation is the backbone of mobility in any city. A transit agency has to decide which stops to connect, how often vehicles should run on each route, and how many buses and drivers it can afford to put on the road. In many places these decisions are still taken using historical routes, manual spreadsheets, and the experience of a few planners. Such an approach is slow to adapt when passenger demand shifts, when new residential areas appear, or when the fleet size or the operating budget changes.

The Public Transit Planner replaces this manual process with a data driven pipeline. Raw data about stops, passenger demand, and travel times is collected and cleaned, the road network is modelled as a weighted graph, and graph algorithms are used to propose efficient routes. The system then converts the chosen routes into a practical timetable and measures the quality of the plan with transparent, quantitative metrics.

1.2 Problem Statement

Given a set of transit stops, the travel time between connected stops, the passenger demand between stops, a fixed number of vehicles, and a fixed operating budget, the system must decide which routes to operate and how frequently, such that service coverage is maximized while passenger travel time and operating cost are kept as low as possible. The plan must also be turned into a timetable that respects vehicle availability, and it must be evaluated so that different planning scenarios can be compared objectively.

1.3 Scope of the Project

The project covers the complete planning workflow: loading and cleaning transit data; building a weighted transit graph; computing shortest travel paths; generating candidate routes from demand patterns; selecting routes under fleet and budget limits; assigning headways (the time gap between consecutive buses); generating daily timetables; computing performance metrics; and presenting interactive reports and visualizations for scenario comparison. Real-time vehicle tracking, fare collection, ticketing, and live GPS integration were treated as outside the scope of this submission.

1.4 Objectives

- Clean and validate transit stop, passenger demand, and travel time data so that every later stage works on consistent input.
- Model the transit network as a weighted graph and compute shortest travel times between stops efficiently.
- Select routes that maximize service coverage while minimizing passenger travel time and operating cost.
- Respect the fixed fleet size and fixed budget as hard constraints during planning.
- Generate practical timetables based on demand patterns and vehicle availability.
- Evaluate every plan using route coverage, average waiting time, travel time, fleet utilization, and operational expenses.
- Provide interactive reports and visualizations to compare planning scenarios and support decision making.

1.5 Importance of the Application

Better planning directly improves accessibility and service quality. Passengers benefit from shorter waiting and travel times and from more of the city being reachable by public transport. Agencies benefit because the system makes the trade-offs explicit: for example, how much coverage is gained by an additional route, or how much a larger fleet reduces waiting time. Because every plan is expressed as numbers and charts, decisions can be justified and revisited when conditions change.

The project is also an applied study of data structures and algorithms. Adjacency lists, priority queues, hash maps, sorting, and greedy selection each play a concrete role, and the choice of each structure is justified by the time complexity it provides.

1.6 Target Users / Audience

The system is intended for transit planners and municipal transport authorities who need to design or revise routes and timetables; for students and researchers studying transportation optimization; and for decision makers who need clear, comparable summaries of different planning scenarios rather than raw data tables.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The planner works on city-scale datasets of a few hundred stops and a few thousand demand records, so its hardware requirements are modest. A development or demonstration machine with the following configuration is sufficient to run every stage of the pipeline and the interactive dashboard on a single computer.

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core, 2.5 GHz or higher
Memory (RAM)	4 GB	8 GB
Free disk space	500 MB	1 GB
Display	1366 × 768	1920 × 1080
Network	Not required for planning	Needed only to install packages

2.2 Software Requirements

- Operating system: Windows 10/11, Ubuntu 20.04 or later, or macOS
- Python 3.10 or later, with pip
- A modern web browser (Chrome, Edge, or Firefox) for the interactive reports
- Git, for version control
- Visual Studio Code or any Python-capable editor (recommended)

2.3 Development Tools and Libraries

The project is written in Python. The pandas and NumPy libraries are used for data loading, cleaning, and numerical work; the graph algorithms are implemented directly with Python's built-in dictionaries, lists, and the heapq priority queue so that the underlying data structures are visible and explainable, with NetworkX used to cross-check results and to draw network diagrams. Matplotlib and Plotly produce the charts, and Streamlit provides the interactive dashboard. Git and GitHub are used for source control, and Visual Studio Code is the main development environment.

Tool / Library	Purpose
Python 3	Core implementation language
pandas, NumPy	Data loading, cleaning, aggregation, numerical computation
heapq, collections	Priority queue and hash-based structures for the algorithms
NetworkX	Validation of path results and network drawing
Matplotlib, Plotly	Static and interactive charts
Streamlit	Interactive dashboard for scenario comparison
Git / GitHub	Version control

CHAPTER 3: TECHNOLOGY STACK

3.1 Programming Language

Python was chosen because it combines readable syntax with a very strong ecosystem for data analysis. The same language is used for data cleaning, algorithm implementation, metric computation, and the dashboard, which keeps the whole project in one codebase that is easy to read, explain, and modify.

3.2 Data Processing Libraries

pandas reads the CSV inputs into data frames and performs the cleaning steps: removing duplicates, handling missing values, validating ranges, and checking that every edge and demand record refers to a stop that exists. NumPy supports vectorized calculations such as demand aggregation by time period and the computation of averages used in the metrics.

3.3 Graph and Optimization

The transit network is stored as an adjacency list: a dictionary that maps each stop to the list of its neighbouring stops and the travel time to each. This representation uses $O(V + E)$ memory and allows the neighbours of a stop to be scanned in time proportional to their number. Shortest paths are computed with Dijkstra's algorithm using a binary heap (heapq), giving $O((V + E) \log V)$ running time. Route selection uses a greedy strategy that ranks candidate routes by the demand they newly cover per unit of cost, which is a standard and efficient heuristic for budget-constrained selection problems.

Task	Data structure / algorithm	Complexity
Network storage	Adjacency list (dict of lists)	$O(V + E)$ space
Shortest travel time	Dijkstra with binary heap	$O((V + E) \log V)$
Stop lookup	Hash map (dict)	$O(1)$ average
Candidate ranking	Sorting by benefit / cost ratio	$O(R \log R)$
Demand aggregation	Hash-based grouping	$O(N)$ for N records

3.4 Visualization and Reporting

Matplotlib is used for static figures such as the network drawing and metric bar charts. Plotly produces interactive charts that support hovering and zooming, and Streamlit assembles them into a browser-based dashboard where the user can change the fleet size or budget and immediately compare the resulting metrics across scenarios.

3.5 Version Control

Git was used throughout development, with the source code, sample datasets, and documentation kept in a single repository hosted on GitHub. Regular commits made it possible to track each stage of the pipeline (data cleaning, graph construction, optimization, timetable generation, and reporting) and to return to a working state whenever an experiment did not succeed.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a layered architecture in which data flows in one direction, from raw input to final reports. Each layer has a single responsibility and communicates with the next only through well-defined data structures. Figure 4.1 shows the four layers.

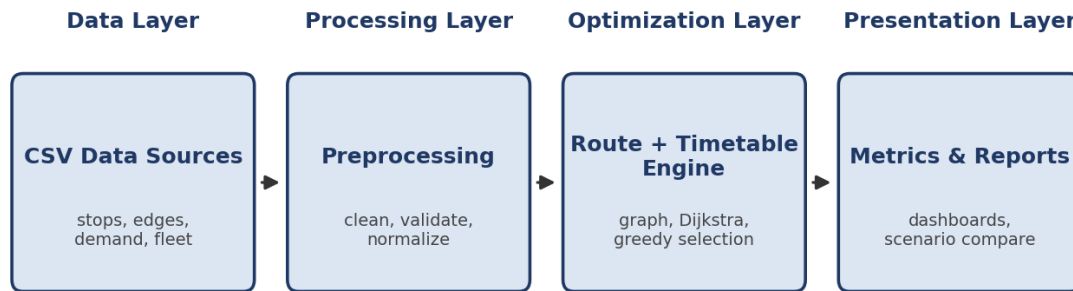


Figure 4.1: Layered architecture of the Public Transit Planner

4.2 Description of Each Layer

Data Layer. Holds the raw inputs as CSV files: the list of stops, the connections (edges) between stops with distance and travel time, the passenger demand between stop pairs for each time period, and the fleet and cost parameters such as number of buses, bus capacity, and cost per kilometre.

Processing Layer. Cleans and validates the inputs. Duplicate stops are removed, missing values are handled, impossible values (for example a negative travel time) are rejected, and every reference between files is checked so that the later stages can assume consistent data.

Optimization Layer. Builds the weighted graph, computes shortest paths, generates candidate routes from high-demand stop pairs, selects the best set of routes under the budget and fleet limits, assigns headways, and generates the timetable.

Presentation Layer. Computes the evaluation metrics and presents them through tables, charts, and the interactive dashboard so that planning scenarios can be compared side by side.

4.3 Processing Pipeline

Figure 4.2 shows the end-to-end pipeline. Every stage consumes the output of the previous stage and can be run and tested on its own, which made incremental development and debugging straightforward.

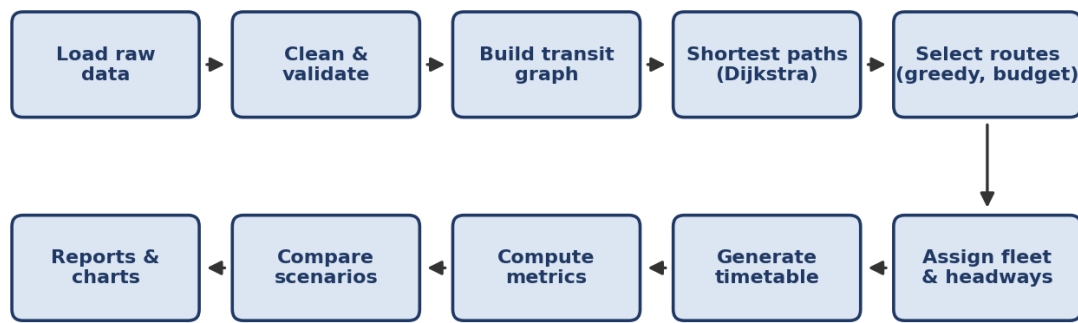


Figure 4.2: End-to-end processing pipeline

4.4 Deployment Architecture

For development and demonstration, the whole system runs as a single local Python process on one machine. The dashboard is served locally by Streamlit and opened in the browser, and all data is read from local CSV files. No database server, container, or cloud service was required for this submission; Chapter 11 outlines how the architecture could be extended to a hosted environment.

CHAPTER 5: DESIGN

5.1 Data Design Principles

The input data is kept in separate, simple tables, and each fact is stored in exactly one place. Stops are identified by a unique `stop_id`; every edge, demand record, and route refers to stops only through that identifier. This avoids duplicated information and makes consistency checks straightforward: any record that mentions an unknown `stop_id` is rejected during preprocessing instead of causing a failure later in the pipeline.

5.2 Data Model

The logical data model has six entities, shown in Figure 5.1. **STOPS** describes the physical stops. **EDGES** describes direct road connections between stops with distance and travel time. **DEMAND** records how many passengers want to travel between an origin and a destination in each time period. **FLEET** stores the vehicle count, capacity, and cost parameters. **ROUTES** is produced by the planner and stores the ordered list of stops, the length, and the round-trip time of each chosen route. **TIMETABLE** is also produced by the planner and stores the departure times and stop times of every trip.

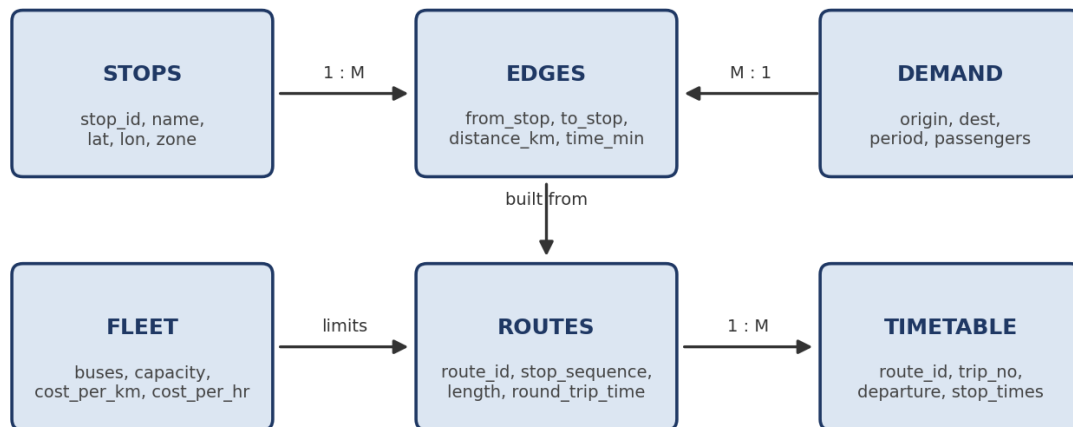
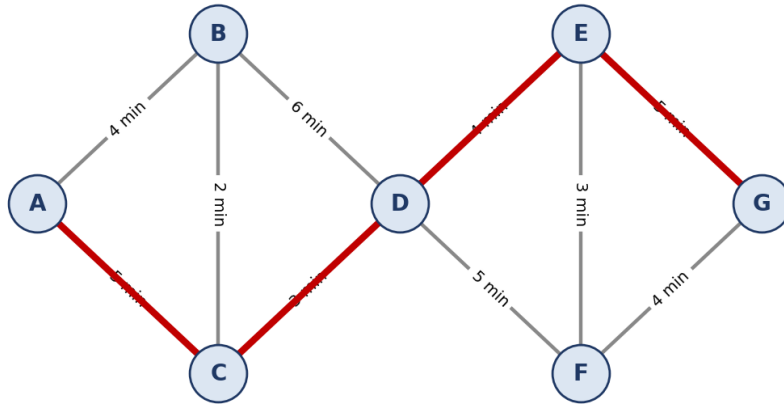


Figure 5.1: Logical data model of the planner

5.3 Algorithm Design

Shortest travel time. The network is an undirected weighted graph in which edge weights are travel times in minutes. Because all weights are non-negative, Dijkstra's algorithm gives exact shortest paths. Figure 5.2 illustrates the idea on a small sample network.

Figure 5.2: Illustrative sample network (red = shortest path A to G)



Candidate route generation. Origin–destination pairs are sorted by demand. For each high-demand pair, the shortest path between the two stops is taken as a candidate route, and nearly identical candidates are merged. Every candidate receives a length, a round-trip time, a demand-covered value, and an operating cost.

Route selection. Candidates are ranked by the ratio of newly covered demand to cost. The planner repeatedly picks the best-ranked candidate whose cost fits in the remaining budget and whose bus requirement fits in the remaining fleet, and updates the covered demand after every pick so that overlapping routes are not double counted.

Headway and fleet allocation. For a route r , the number of trips per hour needed is $f_r = \text{ceil}(\text{peak demand} / (\text{bus capacity} \times \text{target load factor}))$, and the headway is $h_r = 60 / f_r$ minutes. If a round trip takes T_r minutes, the buses needed on the route are $N_r = \text{ceil}(T_r / h_r)$.

5.4 Constraints and Objective

The planner works under two hard constraints and one combined objective:

- **Fleet constraint:** the sum of N_r over all selected routes must not exceed the total number of available buses.
- **Budget constraint:** the total daily operating cost, computed from distance run and bus-hours used, must not exceed the available budget.
- **Objective:** maximize the share of demand covered by the selected routes while minimizing demand-weighted travel time and operating cost.

5.5 Design Decisions and Assumptions

Several design decisions were made explicitly and are recorded here rather than left implicit in the code:

- **Own implementation of core algorithms.** Dijkstra's algorithm and the greedy selection are implemented directly instead of calling library functions, so that the data structures and complexity can be explained; NetworkX is used only to cross-check results.
- **Waiting time model.** Passengers are assumed to arrive uniformly at random, so the average waiting time on a route is half of its headway.
- **Demand aggregation.** Demand is aggregated into peak and off-peak periods so that headways can differ during the day while keeping the model simple.

- **Fixed fleet and budget.** Both are treated as hard limits that the planner may not exceed; it reduces service or drops routes instead.
- **Static data.** Travel times are treated as fixed averages per time period; real-time traffic variation is not modelled.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The code is organized into small modules, each responsible for one stage of the pipeline. Figure 6.1 shows how the entry script coordinates them.

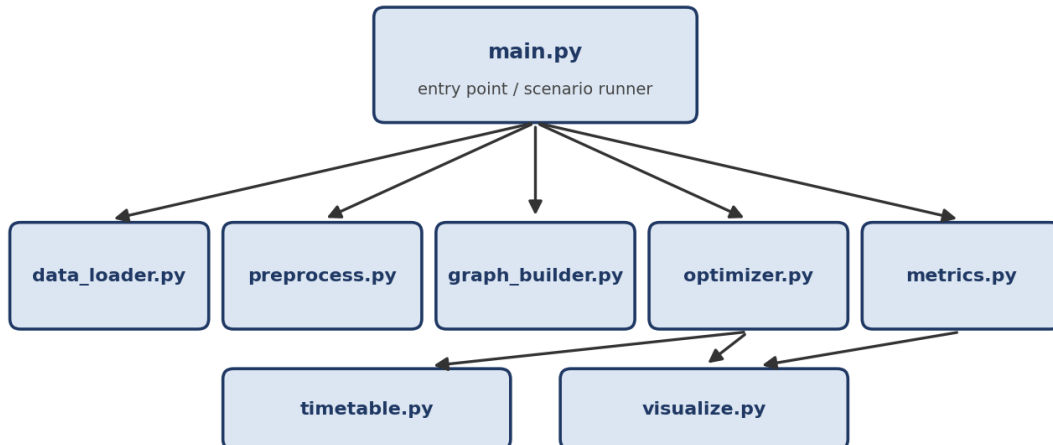


Figure 6.1: Module structure of the implementation

Module	Responsibility
data_loader.py	Reads the CSV inputs and the fleet / cost configuration
preprocess.py	Cleaning, validation, and consistency checks
graph_builder.py	Builds the adjacency list and runs Dijkstra's algorithm
optimizer.py	Candidate routes, greedy selection, headway and fleet allocation
timetable.py	Departure and stop time generation
metrics.py	Coverage, waiting time, travel time, utilization, and cost
visualize.py	Charts and the interactive dashboard

6.2 Data Preprocessing

Preprocessing runs before any algorithm and applies the following rules in order: (1) remove exact duplicate records; (2) drop or fill records with missing mandatory fields, logging every change; (3) reject non-positive distances and travel times; (4) verify that every edge endpoint and every demand origin and destination exists in the stops table; (5) remove self-loops; and (6) check connectivity of the graph and report any isolated stops. The cleaned tables are the only input that later stages are allowed to read, which guarantees that the algorithms never see inconsistent data.

6.3 Graph Construction and Routing

The cleaned edges are converted into an adjacency list. Each undirected edge is inserted in both directions with its travel time as the weight. Dijkstra's algorithm uses a binary heap as a priority queue; a popped entry is ignored if it is stale, meaning a shorter distance to that stop has already been found. After the algorithm finishes, the shortest path to any stop is rebuilt by following the stored predecessor links backwards. To avoid recomputation, shortest-path results from each frequently used source stop are cached in a dictionary.

Pseudocode of the routing step:

```

dijkstra(graph, source):
    dist[v] = infinity for all v; dist[source] = 0
    push (0, source) into min-heap
    while heap is not empty:
        (d, u) = pop minimum
        if d > dist[u]: continue          # stale entry
        for (v, w) in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w; prev[v] = u
                push (dist[v], v)
    return dist, prev

```

6.4 Route Selection and Fleet Allocation

Candidate routes are produced from the highest-demand origin–destination pairs. For each candidate the planner computes its length, round-trip time, the set of stops it serves, and its demand value. The greedy loop then repeatedly (a) recomputes the marginal benefit of every remaining candidate, which is the demand it would cover that is not yet covered, (b) selects the candidate with the highest benefit per unit cost that still fits in the remaining budget and fleet, and (c) stops when no candidate fits or no candidate adds coverage. Headways are then assigned from the peak demand of each route using the formula in Chapter 5, and the buses needed per route are summed to confirm that the fleet constraint holds. If the fleet is exceeded, the lowest-value route is dropped or its headway is lengthened until the plan is feasible.

6.5 Timetable Generation

For each selected route the timetable generator uses the service window (by default 06:00 to 22:00), the peak and off-peak headways, and the cumulative travel time along the route. The first departure is placed at the start of the service window, and each following departure is the previous one plus the headway for that time of day. The arrival time at every stop is the departure time plus the cumulative travel time to that stop, so a complete stop-by-stop schedule is produced. A check at the end confirms that the number of buses simultaneously in service never exceeds the fleet size.

6.6 Metrics Computation

Metric	Definition
Route coverage	Percentage of total passenger demand whose origin and destination are both served by the selected routes
Average waiting time	Demand-weighted mean of half the headway ($h / 2$) on the routes used
Average travel time	Demand-weighted mean of in-vehicle time plus transfer penalties along the shortest usable path
Fleet utilization	Bus-hours actually used divided by bus-hours available, expressed as a percentage
Operating expense	Sum over routes of (distance run $\times$ cost per km) + (bus-hours $\times$ cost per hour)

6.7 Visualization Logic

The visualization module reads the metrics of one or more scenarios and produces comparison tables and charts: a bar chart of metrics side by side, a network drawing with the selected routes highlighted, and a timetable view for a chosen route. In the dashboard, the user can change the fleet size or the budget; the planner is re-run for the new values and the new scenario is added to the comparison, which lets a decision maker see immediately how a change affects coverage, waiting time, and cost.

CHAPTER 7: FEATURES

7.1 List of Main Features

- Automatic cleaning and validation of stop, demand, and travel time data, with a log of every correction
- Weighted transit graph construction and fast shortest-path computation between any two stops
- Route generation and selection that respects a fixed fleet size and a fixed operating budget
- Demand-based headway assignment with separate peak and off-peak service
- Automatic daily timetable generation for every selected route, with stop-by-stop times
- Five performance metrics: route coverage, average waiting time, average travel time, fleet utilization, and operating expense
- Scenario comparison: several plans with different fleet sizes or budgets compared side by side
- Interactive charts and a dashboard for exploring the results

7.2 How Each Feature Works

Data cleaning. Every input table passes through the validation rules of Section 6.2. The outcome is a cleaned data set plus a short report listing how many records were removed, repaired, or rejected, so the planner always knows how trustworthy the input was.

Shortest path lookup. Given two stops, the system runs Dijkstra's algorithm from the origin and reads the distance and path to the destination. Results are cached, so repeated queries for the same origin are answered instantly.

Constrained route selection. The planner keeps track of the remaining budget and remaining buses while it selects routes. A route is only added if both limits still allow it, so the final plan can never violate the constraints.

Timetable generation. The timetable is generated from the headway and the cumulative travel time along the route. For example, with a first departure at 06:00 and a 15-minute headway, the trips leave at 06:00, 06:15, 06:30, and so on until the end of the service window.

Scenario comparison. Each run of the planner is stored as a scenario containing its routes, timetable, and metrics. The dashboard shows two or more scenarios in one table and one chart, and highlights which scenario is best for each metric.

Interpreting the metrics. A good plan has high coverage, low waiting and travel times, utilization close to (but not above) 100 percent, and cost within budget. Because improving one metric often worsens another (for example, shorter headways reduce waiting time but raise cost), the comparison view is essential for choosing a balanced plan.

CHAPTER 8: TESTING

8.1 Testing Strategy

Testing followed the structure of the pipeline. Each module was tested on its own with small, hand-checkable inputs before it was connected to the next stage, and the complete pipeline was then run on the full sample data set. Small networks such as the sample network in Figure 5.2 were especially useful because the correct shortest path and its length can be verified by hand.

8.2 Unit and Integration Testing

Unit tests target individual functions: the cleaning rules, Dijkstra's algorithm, the headway and bus-count formulas, the timetable generator, and each metric. **Integration tests** run the whole pipeline from raw files to final metrics and check global properties that must always hold: the plan never exceeds the fleet or the budget, every timetable departure lies inside the service window, coverage lies between 0 and 100 percent, and utilization never exceeds 100 percent. The shortest-path results were also cross-checked against NetworkX on several networks.

8.3 Tools Used for Testing

- Python unittest / pytest — automated tests of individual functions
- NetworkX — independent cross-check of shortest-path results
- pandas — inspection of cleaned data and intermediate tables
- Manual dashboard testing in the browser — checking charts, scenario comparison, and controls

8.4 Test Cases

The following test cases were designed to cover data validation, routing, optimization, timetabling, and metrics. For each case the expected result, which is also its pass criterion, is stated.

#	Test Case	Expected Result
1	Stop record with a missing stop name	Record flagged, repaired or dropped, and listed in the cleaning report
2	Duplicate stop_id in the stops file	Only the first record is kept; duplicate count reported
3	Edge with a negative or zero travel time	Edge rejected during preprocessing
4	Edge or demand record referring to an unknown stop	Record rejected; no crash in later stages
5	Network that is not fully connected	Isolated stops reported; planner warns that they cannot be served
6	Shortest path from A to G on the sample network (Figure 5.2)	Minimum travel time of 17 minutes
7	Shortest path from a stop to itself	Path contains only that stop; travel time 0
8	Budget set to zero	No routes selected; clear message shown
9	Very large budget, small fleet	Selection stops when the fleet limit is reached; buses used $\leq$ fleet size
10	Fleet smaller than the buses a route needs	Headway lengthened or route dropped until the plan is feasible
11	Timetable: first departure 06:00, headway 15 min, three trips	Departures at 06:00, 06:15, 06:30

#	Test Case	Expected Result
12	Average waiting time for a route with a 20-minute headway	10 minutes
13	Fleet utilization for any generated plan	Never above 100 percent
14	Two scenarios with different budgets	Comparison table shows metrics of both scenarios side by side

CHAPTER 9: DEPLOYMENT

9.1 Steps to Run

The application runs locally for development and demonstration as follows:

- Install Python 3.10 or later and clone the project repository.
- Create and activate a virtual environment, then install the dependencies with `pip install -r requirements.txt`.
- Place the input CSV files (stops, edges, demand) and the fleet configuration in the `data/` folder.
- Run the planner from the command line, or start the dashboard with `streamlit run app.py`.
- Open the local address shown in the terminal (usually `http://localhost:8501`) in a browser.

9.2 Environment Configuration

All adjustable parameters are kept in one configuration file instead of being hard-coded: the number of buses, bus capacity, target load factor, cost per kilometre, cost per bus-hour, the daily budget, the service window, the peak and off-peak hours, and the transfer penalty. Changing a value in this file (or in the dashboard controls) is enough to create a new planning scenario without touching the code. A virtual environment keeps the library versions of this project separate from other Python projects on the same machine.

9.3 Hosting

For this submission the planner and dashboard run as local processes, which is appropriate for development and in-class demonstration. A public deployment was not required by the course. Because the dashboard is a standard Streamlit application, it could later be hosted on a cloud platform with the same codebase; this is discussed in Chapter 11.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Inconsistent input data:** stops with missing names, duplicated records, and edges pointing to stops that did not exist caused errors deep inside the algorithms.
- **Competing objectives:** maximizing coverage, minimizing travel time, and minimizing cost pull in different directions, so no single plan is best on every metric.
- **Repeated shortest-path computation:** running Dijkstra's algorithm from every stop for every candidate made early versions slow.
- **Double counting of demand:** overlapping routes initially counted the same passengers twice, inflating the coverage figure.

10.2 Solutions Applied

The input problem was solved by moving all validation into one preprocessing stage, so that the algorithms only ever receive consistent data. For the competing objectives, the greedy selection uses a single benefit-per-cost ratio, and the scenario comparison view lets the decision maker see the trade-offs instead of hiding them behind one number. Repeated shortest-path computation was reduced by caching the result for each source stop. Double counting was fixed by maintaining a set of already covered demand pairs and recomputing each candidate's marginal benefit after every selection.

10.3 Current Limitations

- The greedy selection is a heuristic: it is fast and gives good plans, but it does not guarantee the mathematically optimal route set.
- Travel times and demand are static averages; real-time traffic, delays, and special events are not modelled.
- Waiting time assumes uniformly random passenger arrivals, which is an approximation.
- The system plans routes and timetables but does not model fares, driver shifts, or vehicle maintenance schedules.
- The system has been run only as a local application and has not been deployed to a public cloud environment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Integration of real-time and historical traffic data so that travel times vary by time of day.
- Support for multiple vehicle types (for example, mini-buses and articulated buses) with different capacities and costs.
- Driver shift planning and vehicle scheduling on top of the generated timetable.
- Transfer-aware planning that optimizes connections between routes explicitly.
- A map-based interface that draws the selected routes over real city maps.

11.2 Possible Integrations or Optimizations

- Adoption of the open GTFS (General Transit Feed Specification) format for importing and exporting data, so the planner can use real agency data and feed standard journey-planning apps.
- Replacing or complementing the greedy selection with metaheuristics such as genetic algorithms or simulated annealing to search for better route sets.
- Use of the A* algorithm with geographic distance as a heuristic to speed up large-scale routing.
- Demand forecasting with machine learning to plan for future rather than past passenger patterns.
- Hosting the dashboard on a cloud platform and packaging the application with Docker for reproducible deployment.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace manual, experience-based transit planning with a transparent, data driven system. Transit stop, demand, and travel time data are cleaned and validated; the network is modelled as a weighted graph; shortest paths and a greedy, constraint-aware selection produce efficient routes; and demand-based headways are converted into a practical timetable. The resulting plan is evaluated with five clear metrics and presented through interactive reports that allow different scenarios to be compared.

12.2 What Was Achieved

The finished planner takes raw transit data and produces a complete, constraint-respecting plan: a set of routes, the headways and fleet allocation for each, and a full timetable. It evaluates every plan on route coverage, average waiting time, average travel time, fleet utilization, and operating expense, and it lets the user compare scenarios side by side. In this way it demonstrates how data preprocessing, optimization algorithms, and data analytics can be combined to address a real-world transportation challenge, improving accessibility and service efficiency while remaining cost effective.

12.3 Skills Learned

Working on this project gave practical experience in cleaning and validating real-world style data; modelling a real problem as a weighted graph; implementing and analysing Dijkstra's algorithm with a priority queue; designing greedy heuristics under multiple constraints; reasoning about time and space complexity when choosing data structures; defining meaningful performance metrics; building interactive visualizations; and testing a pipeline stage by stage so that errors are found close to where they arise. It also showed in a concrete way how algorithmic ideas from the course apply outside the classroom.

CHAPTER 13: REFERENCES

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press (chapters on graph algorithms, shortest paths, and greedy algorithms).
- E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- Python — Official Documentation (including the `heapq` and `collections` modules), <https://docs.python.org/3/>
- pandas — Official Documentation, <https://pandas.pydata.org/docs/>
- NumPy — Official Documentation, <https://numpy.org/doc/>
- NetworkX — Official Documentation, <https://networkx.org/documentation/stable/>
- Matplotlib — Official Documentation, <https://matplotlib.org/stable/>
- Plotly Python — Official Documentation, <https://plotly.com/python/>
- Streamlit — Official Documentation, <https://docs.streamlit.io>
- General Transit Feed Specification (GTFS) — Reference, <https://gtfs.org>
- Git — Official Documentation, <https://git-scm.com/doc>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Dijkstra's shortest-path algorithm with a binary heap, the core routing routine used throughout the planner:

```
import heapq

def dijkstra(graph, source):
    """graph: {node: [(neighbour, travel_time_min), ...]}"""
    dist = {n: float("inf") for n in graph}
    prev = {n: None for n in graph}
    dist[source] = 0
    pq = [(0, source)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue # stale queue entry
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v], prev[v] = nd, u
                heapq.heappush(pq, (nd, v))
    return dist, prev

def build_path(prev, target):
    path = []
    while target is not None:
        path.append(target); target = prev[target]
    return path[::-1]
```

Appendix B: Installation and Setup Instructions

- git clone <repository-url> and cd into the project folder
- python -m venv venv, then activate it (venv\Scripts\activate on Windows, source venv/bin/activate on Linux/macOS)
- pip install -r requirements.txt
- Place stops.csv, edges.csv, demand.csv and the fleet configuration in the data/ folder
- streamlit run app.py, then open the displayed local address in a browser

Appendix C: Input Data Dictionary

File	Columns	Description
stops.csv	stop_id, name, lat, lon, zone	One row per transit stop
edges.csv	from_stop, to_stop, distance_km, travel_time_min	Direct connection between two stops (undirected)
demand.csv	origin, destination, period, passengers	Passengers per period travelling between a stop pair
fleet config	buses, capacity, load_factor, cost_per_km, cost_per_hr, budget	Fleet and cost parameters for a scenario

Appendix D: Module Reference Summary

Module	Main functions
data_loader.py	load_stops, load_edges, load_demand, load_config
preprocess.py	remove_duplicates, validate_edges, validate_demand, check_connectivity
graph_builder.py	build_graph, dijkstra, build_path
optimizer.py	generate_candidates, select_routes, assign_headways, check_fleet
timetable.py	generate_timetable, stop_times
metrics.py	coverage, avg_waiting, avg_travel, utilization, operating_cost
visualize.py	plot_network, plot_metrics, compare_scenarios

Appendix E: Screenshots

The following pages are left for the students to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

1. Dashboard home with scenario controls

[Insert screenshot: Dashboard home with scenario controls]

2. Data cleaning report

[Insert screenshot: Data cleaning report]

3. Transit network with selected routes highlighted

[Insert screenshot: Transit network with selected routes highlighted]

4. Shortest path between two stops

[Insert screenshot: Shortest path between two stops]

5. Generated timetable for a route

[Insert screenshot: Generated timetable for a route]

6. Metrics table for one scenario

[Insert screenshot: Metrics table for one scenario]

7. Scenario comparison chart

[Insert screenshot: Scenario comparison chart]

8. Fleet utilization and cost chart

[Insert screenshot: Fleet utilization and cost chart]